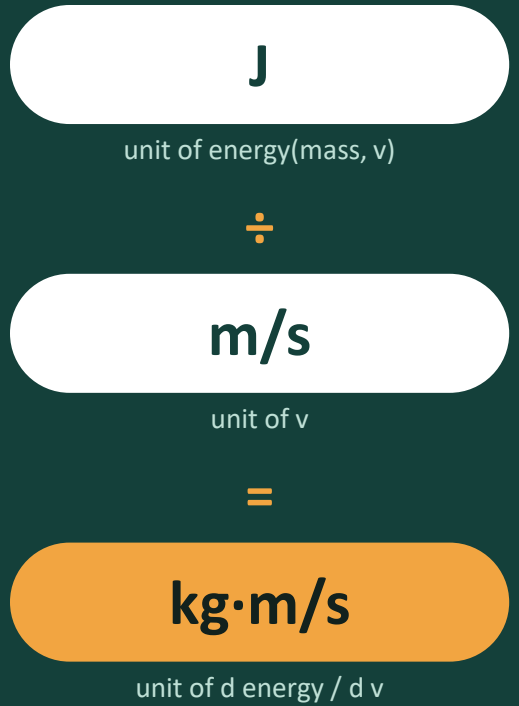
DimGrad

A source-to-source compiler that infers and checks the units of derivatives and emits C

Samartha Kashyap · 24BCE2702
Kaavya Dekhane · 24BCT0077

Compiler Design · Project Review 1



The problem

A unit mistake compiles and runs silently

1999

Mars Climate Orbiter lost

NASA's investigation board named the root cause: English units were not translated into metric in ground navigation software.

1. **Numbers carry no units**

   To the compiler, metres and feet are both just a double.

2. **Derivatives carry units too**

   dE/dv is a momentum. A learning rate needs the unit $[x]^2/[f]$.

3. **Autodiff ignores them**

   Gradient tools compute derivatives with no unit check at all.

Objectives

What the compiler must do, and how we test it

1 Language

A small imperative language with unit annotations

2 Unit checking

Infer and check the unit of every expression

3 Differentiation

Forward mode; each derivative gets the unit [f]/[x]

4 Update checks

Check Newton, descent and Euler steps; solve ? units

5 C output

Emit C for each function and its gradient

6 Evaluation

Injected unit bugs, finite differences, code speed

Literature survey

Two mature fields that rarely meet

Area	Representative work	Missing for this problem
Unit checking	Kennedy's dimension types (F#), Osprey for C, unit inference for Fortran	**No differentiation**
Autodiff	Tapenade, Clad, Tangent, Enzyme	**No units**
Units + autodiff	mp-units, unxt (JAX), ForwardDiff with Unitful	**Libraries; derivative units given by hand**
Theory	2026 preprint on dimensional types; unit analysis of learning rates	**No autodiff implementation**
Adjacent	NEURON modlunit, Pasado	**ODE states only, or no units**

Gap and novelty

Novelty

The idea is known; a compiler that does it is not

THE GAP

No open, evaluated source-to-source compiler statically infers derivative units for general functions and emits C.

Not claimed: that a derivative's unit is a quotient of units. That is prior work.

WHAT IS NEW

- **Static inference**
  Derivative units are computed, not declared by hand
- **Solved placeholders**
  A ? unit is solved by Gaussian elimination
- **Verified tangents**
  The unit of every generated tangent is re-derived
- **Fractional exponents**
  Exact end to end, such as volatility in 1/yr^(1/2)

From source to unit

Implementation

The compiler reports that d energy / d v is a momentum

```
unit kg, m, s;
unit J = kg*m^2/s^2;

fn energy(mass: kg, v: m/s) -> J {
    return 0.5 * mass * v * v;
}

grad energy wrt v;
```

COMPILER REPORT

```
fn energy(mass: kg, v: m/s) -> J
grad energy wrt v : kg·m/s
```



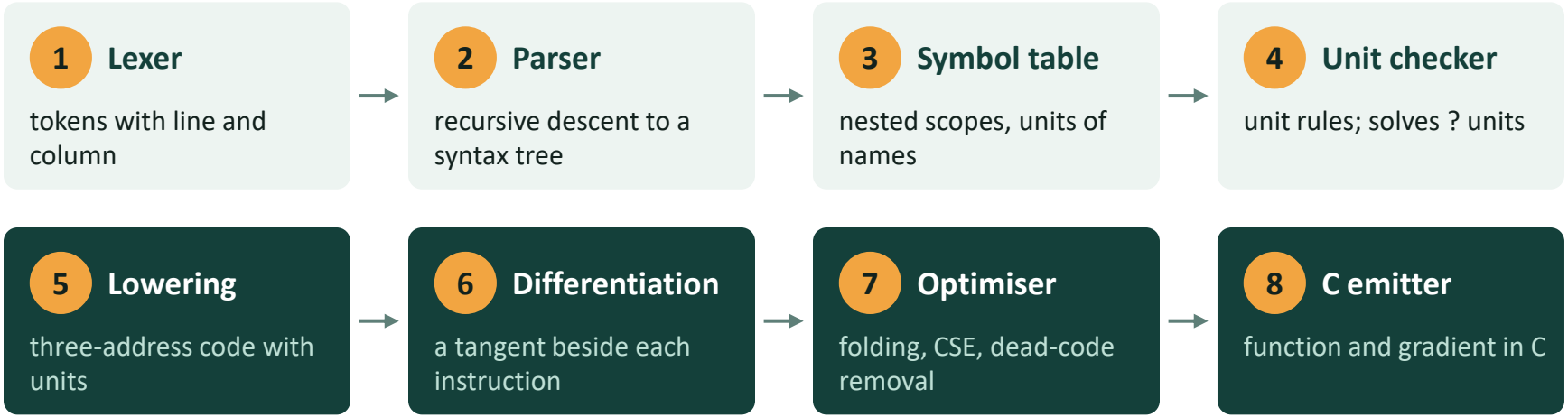
energy ÷ speed = momentum

The same unit check then runs on the generated code: both terms of every tangent rule must agree.

Compiler pipeline

Implementation

Eight stages in Python; GCC turns the emitted C into machine code



551 tests passing

0 third-party packages

12 → 4 instructions in the sample gradient

Checks in action

Implementation

A wrong program is rejected; an unknown unit is solved

REJECTED

```
fn altitude(reading: ft,
            correction: m) -> m {
    return reading + correction;
}
```

error: cannot add ft and m

in expression: reading + correction

Reported with file, line and column.

SOLVED

```
fn gd_step(x: m, eta: ?) -> m {
    return x - eta * f_grad_x(x);
}
```

eta : s^2/kg

The same unit as m^2/J, which is $[x]^2 / [f]$.

Matches the hand derivation for learning rates.

Results so far

Implementation

Measured on 14 benchmark programs and 33 gradients

2.6e-8

largest relative error against finite differences

98.4%

of injected unit errors rejected (182 of 185)

15 / 15

Newton, descent and Euler cases as derived by hand

48%

of instructions removed by the optimiser

- All 3 missed errors were absorbed by a ? parameter that solved to a different unit.
- A gradient costs about 1.22× the function with GCC -O2; timings vary from run to run.

Limits and next steps

Presentation & Q&A

What the first version does not do, and what comes next

LIMITS TODAY

- Scalars only: no arrays, while loops or recursion
- Forward mode, first derivatives only
- No unit polymorphism
- A ? parameter can absorb a unit error
- Angles are treated as unitless

NEXT

- Second derivatives by inlining gradient functions
- Unit polymorphism for reusable functions
- Re-check every cited source and the novelty claim
- Write the measured results into the paper

Demo and questions

Three commands show the checker, an error and the optimised gradient

```
# the unit of the derivative
python -m dimgrad check benchmarks/kinetic_energy.dg

# a unit error, with file, line and column
python -m dimgrad check tests/golden/errors/feet_metres.dg

# three-address code, before and after optimisation
python -m dimgrad compile benchmarks/kinetic_energy.dg --emit-tac --stats
```

J ÷ m/s = kg·m/s

Samartha Kashyap · Kaavya Dehkane